
Ground-Truth-First Benchmarking of Reinforcement Learning via Converse Optimality

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Benchmarking in reinforcement learning (RL) is usually relative: algorithms are
2 ranked by returns on environments whose optimal policies are unknown. This
3 makes distance to optimality unobservable and entangles algorithmic behavior with
4 reward design, stochasticity, hyperparameter choices, and simulator details. We
5 develop a *ground-truth-first* benchmark view based on converse optimality. Rather
6 than introducing a new converse theorem, the paper contributes a unified benchmark
7 methodology and a self-contained empirical synthesis of two converse-optimal
8 benchmark lines: deterministic continuous-time generators for sensitivity studies
9 and stochastic discrete-time generators for oracle-referenced comparison. The first
10 benchmark mode is *sensitivity benchmarking*: one samples nearby environments
11 from a converse generator and asks whether a learning procedure remains consistent
12 under environment variation. The second is *oracle-referenced benchmarking*:
13 one compares learned policies directly to a certified optimum by optimality gap,
14 regret, Bellman residual, and policy error. A key message is that continuous-time
15 and discrete-time generators serve different empirical purposes. Continuous-time
16 generators are natural for physically interpretable sensitivity studies, but their
17 realized behavior depends on the simulator discretization step Δt and numerical
18 scheme. Native discrete-time generators remove this extra degree of freedom:
19 the environment, oracle, and learner all operate on the same transition law, so
20 the benchmark is not confounded by an external integration choice. We make
21 the manuscript empirically self-contained by reproducing representative quantitative
22 results from both settings, including an optimizer-sensitivity case study and
23 oracle-referenced comparisons on serial planar arms and dynamically-extended
24 nonholonomic vehicles.

25 1 Introduction

26 Benchmarking remains one of the main empirical instruments of reinforcement learning (RL),
27 yet standard practice still revolves around learning curves, final returns, and leaderboard-style
28 comparisons [1, 9, 12]. This protocol is useful, but it leaves unanswered the question that matters
29 most for control and decision-making: *how far is the learned policy from optimal?* If the optimum
30 is unknown, a benchmark can show that method A scores higher than method B, but it cannot say
31 whether A is close to optimal, whether both are poor, or whether the ranking is caused by reward
32 scaling, stochastic variance, initialization effects, or simulator conventions.

33 This ambiguity is especially costly in RL because the same algorithm can look strong or weak
34 depending on environment design, reward shaping, evaluation schedules, and hyperparameter choices
35 [1, 9, 10]. Recent critiques therefore argue that benchmarking should not only rank methods, but also
36 expose what conclusions a benchmark actually supports [12, 15]. For RL, this means that a useful
37 benchmark should do at least two things: it should diagnose *sensitivity*—whether a learning procedure

remains reliable when the environment varies—and it should measure *absolute suboptimality* rather than only relative score differences.

Converse optimality is a natural way to obtain both. The core idea is to reverse the usual direction of optimal control. Rather than starting from dynamics and costs and then solving the Bellman or Hamilton–Jacobi–Bellman equation, we start from a desired value function and policy and construct environments for which they are optimal. The benchmark therefore comes with its own oracle. Once the optimum is part of the fixture, benchmark design changes qualitatively: one can measure optimality gap and regret in absolute units, compare policies under matched noise realizations, and run statistical tests over families of generated environments.

The central claim of this paper is that converse optimality supports *two complementary benchmark modes*.

Mode I: sensitivity and robustness benchmarking. A converse generator defines a distribution over environments together with known value functions. One can then sample many nearby tasks and ask whether changing an optimizer, exploration scheme, or architecture alters the distribution of relative suboptimality. This makes environment sensitivity and robustness statistically testable benchmark objects rather than anecdotal observations.

Mode II: oracle-referenced algorithm benchmarking. When the benchmark is exported as a fixed fixture with a known (V^*, π^*) , one can compare any RL algorithm directly to the certified optimum via optimality gap, regret, Bellman residual, and policy error. This turns benchmarking from relative score comparison into calibrated measurement.

A second key message concerns *time formalism*. Deterministic continuous-time converse generators are valuable because they produce physically interpretable systems and clean sensitivity studies. However, when such systems are trained in digital RL pipelines, the effective benchmark depends on the discretization step Δt and numerical integration scheme. Changing Δt changes the transition operator seen by the agent, and can therefore change empirical conclusions. Native discrete-time converse benchmarks remove this confound: the transition law itself is the benchmark object, and the oracle is defined on that same discrete-time map. Thus, continuous-time generators are ideal for probing *numerical and environment sensitivity*, whereas native discrete-time generators are ideal for *absolute algorithm comparison without timestep ambiguity*.

The novelty of the paper is therefore at the *benchmark-design* level, not at the level of a new converse theorem or a new RL algorithm. Concretely, we contribute:

1. a unified benchmark formalization that treats converse-optimal environments as oracle-equipped fixtures;
2. a clarified split between sensitivity/robustness benchmarking and oracle-referenced comparison;
3. a precise explanation of why continuous-time and native discrete-time generators answer different empirical questions;
4. a reproducible evaluation protocol with validation checks and paired common-random-number (CRN) testing; and
5. a self-contained empirical section that reproduces key quantitative evidence from both the continuous-time and discrete-time converse-benchmark papers.

2 Related Work

RL benchmarking has been shaped by large environment suites such as Atari, MuJoCo, Procgen, MiniGrid, bsuite, and D4RL [2, 4–7, 14, 18]. These benchmarks expose important empirical phenomena, but they typically do not provide exact optimal policies or value functions. As a result, the main performance signal remains relative return.

This limitation interacts with the broader reproducibility problems of deep RL. Henderson et al. [9] showed that conclusions can change substantially with implementation and reporting details. Agarwal et al. [1] emphasized that a few runs and point estimates are often insufficient for trustworthy empirical conclusions. Open RL Benchmark improves reproducibility and documentation at scale

[10], but even perfect tracking does not by itself make the optimum known. Without ground truth, one cannot measure optimality gap directly.

Converse optimality offers a way to make optimality part of benchmark design. In deterministic control, converse constructions show how to generate dynamics and costs for which a prescribed value function is optimal [8, 13, 16]. This idea was recently used to generate informative deterministic RL benchmarks and to study sensitivity and robustness through environment families rather than single tasks [19]. More recently, stochastic discrete-time converse constructions were derived for noisy control-affine systems, enabling oracle-referenced evaluation by optimality gap and regret on fixtures with certified optima [11].

This paper synthesizes these strands. The emphasis is not only on generating more environments, but on sharpening the evaluation logic itself. Benchmarks are treated as scientific instruments: they should reveal what changes when the environment varies, what remains invariant, and how far a learned policy is from a known optimum.

3 Benchmarking with Converse Certificates

We start from a discounted Markov decision process

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, c, \gamma),$$

where $P(ds' | s, a)$ is a transition kernel, c is a stage cost, and $\gamma \in (0, 1)$. For a stationary policy π ,

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k c(s_k, \pi(s_k)) \mid s_0 = s \right]. \quad (1)$$

The optimal value function satisfies

$$V^*(s) = \inf_{a \in \mathcal{A}} \left\{ c(s, a) + \gamma \int V^*(s') P(ds' | s, a) \right\}. \quad (2)$$

Definition 1 (Bellman residual). *For any candidate value V , define*

$$A_V(s, a) = c(s, a) + \gamma \int V(s') P(ds' | s, a) - V(s). \quad (3)$$

The next statement is routine, but it is the right benchmark-design rule: a converse-optimal fixture must certify that the residual vanishes on the prescribed policy and is nonnegative elsewhere.

Proposition 1 (Certificate criterion). *Assume the Bellman operators are well defined and contractive on the chosen value class. A pair (V, π) is optimal, i.e. $V = V^* = V^\pi$, if and only if, for all $s \in \mathcal{S}$ and $a \in \mathcal{A}$,*

$$A_V(s, \pi(s)) = 0, \quad (4)$$

$$A_V(s, a) \geq 0. \quad (5)$$

For deterministic systems, P becomes a Dirac measure; for finite MDPs, the integral becomes a sum; for continuous actions, first-order conditions imply the minimizing policy when the Bellman integrand is convex in a .

Definition 2 (Converse-optimal benchmark fixture). *A fixture is a tuple*

$$\mathfrak{B}_\theta = (\mathcal{S}, \mathcal{A}, P_\theta, c_\theta, \gamma, V_\theta^*, \pi_\theta^*, \Omega_\theta),$$

where Ω_θ certifies Proposition 1. During training, the agent interacts only with $(\mathcal{S}, \mathcal{A}, P_\theta, c_\theta, \gamma)$. During evaluation, $(V_\theta^*, \pi_\theta^*)$ serves as an oracle.

If θ is random, then $\mathfrak{B}_\theta$ defines a *distribution* of certified environments for sensitivity and robustness studies. If θ is fixed, then $\mathfrak{B}_\theta$ defines a *single oracle-backed fixture* for absolute comparison.

118 4 Continuous-Time and Discrete-Time Generators

119 4.1 Continuous-time converse generators: natural for sensitivity studies

120 Consider a control-affine continuous-time system

$$\dot{s} = f(s) + g(s)a, \quad s \in \mathbb{R}^n, \quad a \in \mathbb{R}^m. \quad (6)$$

121 Let $V \in C^1$ be positive definite. For smooth fields $H : \mathbb{R}^n \rightarrow \mathbb{R}^n$ and $G : \mathbb{R}^n \rightarrow \mathbb{R}^{n \times m}$, define

$$\begin{aligned} g(s) &= G(s), \\ f(s) &= 1/4 G(s)(L_G V(s))^\top - H(s), \\ c(s, a) &= L_H V(s) + \|a\|_2^2. \end{aligned} \quad (7)$$

122 Then

$$c(s, a) + \nabla V(s)^\top (f(s) + g(s)a) = \|a + 1/2(L_G V(s))^\top\|_2^2,$$

123 so the minimizing policy is

$$a^*(s) = -1/2(L_G V(s))^\top. \quad (8)$$

124 This construction is valuable because it produces physically interpretable nonlinear plants while
 125 preserving analytical optimality. One can parameterize G , H , or V and obtain a rich family of nearby
 126 systems, which is exactly what is needed for environment sensitivity and robustness experiments.
 127 However, the moment a continuous-time environment is used inside a digital RL loop, one must
 128 discretize it. The effective transition operator then depends on the chosen simulator step Δt and
 129 numerical integrator. Two studies that differ only in Δt or in the integration method need not
 130 be solving the same discrete-time MDP. Hence, continuous-time converse benchmarks are best
 131 interpreted as *generators of sensitivity studies*: they expose how learning conclusions shift under both
 132 environment variation and numerical realization.

133 4.2 Native discrete-time generators: essential for absolute comparison

134 For oracle-referenced algorithm comparison, native discrete-time constructions are especially attrac-
 135 tive because the transition map itself is the benchmark object. We consider

$$s_{k+1} = f(s_k) + g(s_k)a_k + w_k, \quad w_k \sim \mathcal{N}(0, \Sigma). \quad (9)$$

136 Assume quadratic stage costs and a quadratic candidate value:

$$q(s) = s^\top Q s, \quad \rho(a) = a^\top R a, \quad V^*(s) = s^\top P s + b, \quad (10)$$

137 with $Q \succeq 0$, $R \succ 0$, $P \succ 0$. Define

$$\begin{aligned} B_\gamma(s) &= R + \gamma g(s)^\top P g(s), \\ H_\gamma(s) &= \gamma(P - \gamma P g(s) B_\gamma(s)^{-1} g(s)^\top P). \end{aligned} \quad (11)$$

138 Then the Bellman integrand is a strictly convex quadratic in a , and the optimal policy is

$$\pi^*(s) = -\gamma B_\gamma(s)^{-1} g(s)^\top P f(s). \quad (12)$$

139 **Proposition 2** (Quadratic–Gaussian converse generator). *If the drift satisfies the energy identity*

$$s^\top (P - Q) s = f(s)^\top H_\gamma(s) f(s), \quad (13)$$

140 *then (12) is the unique optimal policy and additive Gaussian noise contributes only the constant*
 141 *value offset*

$$b = \frac{\gamma}{1 - \gamma} \text{tr}(P \Sigma). \quad (14)$$

142 Equation (13) is the design equation for the benchmark generator. Two convenient parameterizations
 143 are

$$f(s) = H_\gamma(s)^{-1/2} S(s) (P - Q)^{1/2} s, \quad S(s)^\top S(s) = I, \quad (15)$$

$$f(s) = \sqrt{s^\top (P - Q) s} H_\gamma(s)^{-1/2} \frac{\tilde{f}(s)}{\|\tilde{f}(s)\|}, \quad \tilde{f}(s) \neq 0. \quad (16)$$

144 The first uses an orthogonal field $S(s)$ to inject bounded nonlinear coupling while preserving the
 145 energy identity exactly.

146 A particularly useful continuation parameter is control strength:

$$g_p(s) = p g(s), \quad p \in (0, 1].$$

147 As p changes, control authority, conditioning, and effective task difficulty change smoothly, while
 148 optimality remains certified. This homotopy is central for controlled difficulty sweeps.

149 **Why this matters for benchmarking.** For a native discrete-time fixture, changing a simulator
 150 step Δt is no longer an external benchmark decision: the transition law (9) already defines the task.
 151 The learner, the oracle, and the evaluation metrics all operate on the same transition operator. This
 152 is what makes optimality gap and regret intrinsic rather than artifacts of an additional numerical
 153 choice. We do not claim that the Δt confound is always dominant in practice; the point is that native
 154 discrete-time benchmark design removes it by construction.

155 5 Generator Validation and Benchmark Families

156 A benchmark family should not be defined only by a converse formula. It must also be exported as a
 157 validated fixture. Each sampled instance therefore undergoes three checks before release:

- 158 1. **Well-posed action minimization:** verify positive definiteness of $B_\gamma(s)$ on a prescribed test region,
 159 ensuring unique optimal actions and numerically stable evaluation.
- 160 2. **Bellman consistency:** compute the residual

$$\delta(s) = \left| V^*(s) - \left(q(s) + \rho(\pi^*(s)) + \gamma \mathbb{E}[V^*(s^+)] \right) \right|$$

161 on a validation grid and accept the instance only if $\max_s \delta(s) \leq \varepsilon$.

- 162 3. **Closed-loop boundedness:** roll out the certified policy π^* over a fixed horizon and reject instances
 163 whose trajectories exhibit numerical blow-up or leave the admissible test domain.

164 This makes the benchmark export reproducible and auditable, rather than merely symbolic.

165 5.1 Serial planar arm: state-dependent actuation and nonlinear coupling

166 Our first family models a serial n -link planar arm with state

$$s = [\theta_1, \omega_1, \dots, \theta_n, \omega_n]^\top \in \mathbb{R}^{2n}, \quad a = [\tau_1, \dots, \tau_n]^\top \in \mathbb{R}^n.$$

167 The input map is configuration dependent:

$$[g(s)]_{2i,i} = \cos \theta_i, \quad i = 1, \dots, n, \tag{17}$$

168 with all other entries zero. This mimics mechanical advantage that varies with joint angle.

169 We construct the drift via (15):

$$f_p(s) = H_\gamma^{(p)}(s)^{-1/2} S_p(s) (P - Q)^{1/2} s. \tag{18}$$

170 The orthogonal field $S_p(s)$ is implemented as a product of Givens rotations,

$$S_p(s) = \prod_{i=1}^{n-1} G(\omega_i, \omega_{i+1}; \beta_i) \prod_{i=1}^n G(\theta_i, \omega_i; \alpha_i), \tag{19}$$

171 with rotation angles

$$\begin{aligned} \alpha_i &= \alpha_0 p \tanh(\kappa_1 \theta_i \omega_i) + g_0 p \sin \theta_i, \\ \beta_i &= \beta_0 p \tanh(\kappa_2 (\theta_{i+1} - \theta_i)). \end{aligned} \tag{20}$$

172 The $\tanh$ terms create bounded velocity-dependent and neighbor-coupling effects, while the $\sin \theta_i$
 173 term introduces a gravity-like bias. The resulting family is converse-optimal, nonlinear, and scalable
 174 in dimension.

175 5.2 Dynamically-extended nonholonomic vehicle: long horizons and delayed actuation

176 Our second family models a unicycle-like vehicle with dynamic extension. For module i ,

$$s_i = [x_i, y_i, \phi_i, v_i^{(0)}, \dots, v_i^{(r_v-1)}, \omega_i^{(0)}, \dots, \omega_i^{(r_\omega-1)}]^\top,$$

177 and action

$$a_i = [u_{v,i}, u_{\omega,i}]^\top.$$

178 The kinematic part is

$$\begin{aligned} x_{i,k+1} &= x_{i,k} + \tau v_{i,k}^{(0)} \cos \phi_{i,k}, \\ y_{i,k+1} &= y_{i,k} + \tau v_{i,k}^{(0)} \sin \phi_{i,k}, \\ \phi_{i,k+1} &= \phi_{i,k} + \tau \omega_{i,k}^{(0)}, \end{aligned} \tag{21}$$

179 while the velocity chains propagate upward and the control acts only at the top of each chain. After
180 stacking modules, the system takes the form

$$s_{k+1} = f_p(s_k) + pGa_k + w_k, \quad w_k \sim \mathcal{N}(0, \Sigma),$$

181 with block-diagonal constant input matrix G . The drift again uses the metric-normalized converse
182 parameterization

$$f_p(s) = (H_\gamma^{(p)})^{-1/2} S_\nu(s) (P - Q)^{1/2} s, \tag{22}$$

183 where $S_\nu(s)$ is an orthogonal field built from bounded rotations coupling position, heading, and
184 velocity states. This family is useful because dynamic extension creates delayed actuation, longer
185 credit assignment paths, and stiff closed-loop behavior. It therefore stresses RL algorithms differently
186 from the arm family even though both come with certified optima.

187 6 Evaluation Protocols

188 6.1 Mode I: sensitivity and robustness over generated families

189 Let θ index a converse-generated environment. For a policy π and initial state s_0 , define the
190 continuous-time or discrete-time cost $J_\theta(\pi; s_0)$ and normalize it by the known optimum:

$$\text{RS}(\pi; \theta, s_0) = \frac{J_\theta(\pi; s_0)}{V_\theta^*(s_0)}. \tag{23}$$

191 The benchmark then becomes a distributional object: one compares samples of RS over many θ 's
192 and asks whether one learning procedure has worse mean or variance than another. Sensitivity refers
193 to variation *during training*; robustness refers to training on nominal environments and testing the
194 resulting policy on shifted environments.

195 The statistical design is straightforward: sample environments i.i.d. from the generator, train two
196 learning procedures that differ only in the factor under test, compute relative suboptimality for each
197 arm, and apply mean tests, variance tests, or both depending on the scientific question. Because every
198 environment comes with its own optimal value, performance is normalized in a task-aware way rather
199 than by raw return.

200 6.2 Mode II: oracle-referenced evaluation on native discrete-time fixtures

201 For a learned policy $\hat{\pi}$ and horizon T , define

$$J_T(\hat{\pi}; s_0) = \mathbb{E} \left[\sum_{k=0}^{T-1} \gamma^k c(s_k, \hat{\pi}(s_k)) \mid s_0 \right]. \tag{24}$$

202 The normalized optimality gap is

$$\text{OptGap}_T(\hat{\pi}; s_0) = \frac{J_T(\hat{\pi}; s_0) - V^*(s_0)}{|V^*(s_0)| + \varepsilon}, \tag{25}$$

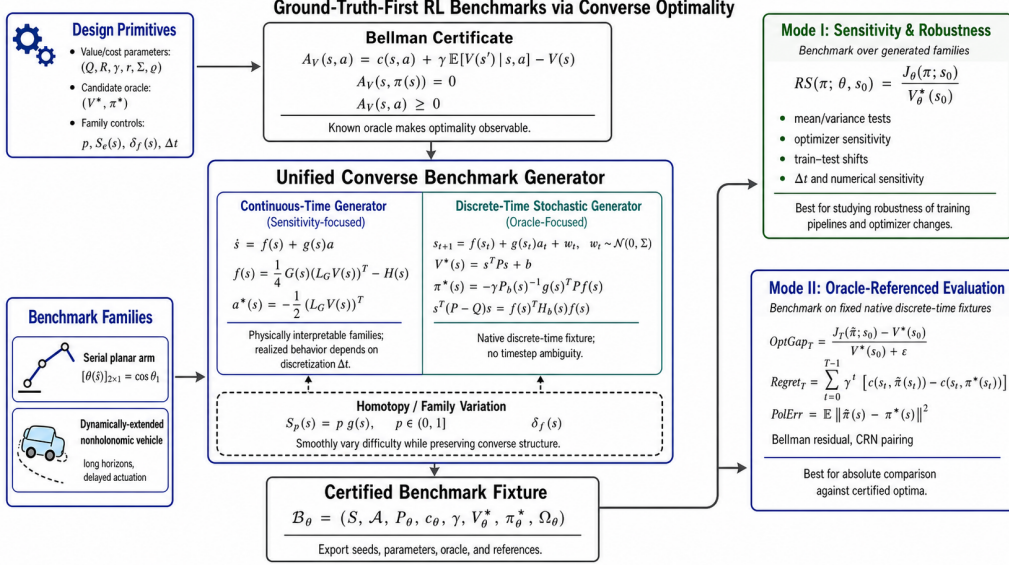


Figure 1: Ground-truth-first benchmark pipeline. A converse generator samples parameters, constructs a certified environment family, validates the certificate, exports fixtures, and supports either sensitivity benchmarking over sampled families or oracle-referenced evaluation on fixed fixtures.

203 and discounted regret is

$$204 \quad \text{Regret}_T(\hat{\pi}; s_0) = \sum_{k=0}^{T-1} \gamma^k \left(c(s_k, \hat{\pi}(s_k)) - c(s_k, \pi^*(s_k)) \right). \quad (26)$$

204 If actions are directly comparable, we also report policy error

$$\text{PolErr} = \mathbb{E}_{s \sim \mu} \|\hat{\pi}(s) - \pi^*(s)\|_2^2.$$

205 To reduce evaluation variance, all algorithm comparisons use *common random numbers* (CRN): for
206 each fixture θ we pre-generate a deterministic evaluation schedule

$$\mathcal{E}_\theta = \{(s_0^{(j)}, w_{0:T-1}^{(j)})\}_{j=1}^M$$

207 of start states and disturbance streams. Every trained policy from every algorithm is evaluated
208 on exactly this same schedule. Confidence intervals are then computed by *paired bootstrap* over
209 trajectory indices j , so the bootstrap preserves the within-trajectory correlation induced by CRN. This
210 is the operational detail that turns oracle-referenced comparison into a paired statistical test rather
211 than a collection of unpaired rollouts.

212 7 Self-Contained Empirical Evidence

213 The paper is meant to stand on its own empirically. We therefore reproduce representative quantitative
214 evidence from both source lines rather than referring vaguely to external plots. The continuous-
215 time results demonstrate the sensitivity-benchmark mode; the discrete-time results demonstrate the
216 oracle-referenced mode.

217 7.1 Continuous-time case study: optimizer sensitivity

218 The first use of the framework is not to decide which RL algorithm “wins” on a single task, but to ask
219 whether a training pipeline is stable under environment variation. In the continuous-time sensitivity
220 study, a converse generator created a sample of unstable three-dimensional control problems and
221 compared two PPO pipelines that differed *only* in their optimizer: Adam versus RAdam. Because
222 each sampled environment came with a known value function, the final performance could be

Table 1: Self-contained reproduction of the continuous-time sensitivity case study. Two PPO pipelines differed only in the optimizer. The benchmark quantity is relative suboptimality over a converse-generated environment sample.

Quantity	Value
Generator family	Unstable 3D continuous-time converse-optimal systems
Learning algorithm	PPO (Stable Baselines)
Perturbation under test	Optimizer only: Adam vs. RAdam
Environment sample size	160 generated problems, split into 80 per optimizer arm
Normalization	Relative suboptimality using the known optimal value for each sampled environment
Test statistic	Variance-ratio $F \approx 2.59$
p -value	$\approx 1.7 \times 10^{-5}$
Interpretation	Replacing Adam with RAdam materially changed the variance of the outcome distribution; optimizer choice affected environment sensitivity, not only average score

Table 2: Overall performance per algorithm and experiment. Metrics are mean and 95% CI via paired bootstrap under CRN. Best values within each experiment are bolded: for OptGap, closest to zero is best; for Regret, smaller is best.

Experiment	Algorithm	OptGap (mean)	OptGap 95% CI	Regret (mean)	Regret 95% CI
Serial-random	A2C	-2.36e+03	[-2.79e+03, -1.93e+03]	8.24e+03	[6.83e+03, 9.58e+03]
Serial-random	DDPG	-2.29e+03	[-3.83e+03, -9.12e+02]	9.28e+03	[3.50e+03, 1.63e+04]
Serial-random	PPO	-3.32e+02	[-8.45e+02, -4.92e+01]	1.51e+03	[1.92e+02, 3.93e+03]
Serial-random	SAC	-1.65e+02	[-3.47e+02, -4.63e+01]	6.90e+02	[1.72e+02, 1.53e+03]
Serial-random	TD3	-5.85e+01	[-8.64e+01, -3.57e+01]	2.14e+02	[1.25e+02, 3.35e+02]
Serial-fixed	A2C	-1.11e+03	[-2.15e+03, -4.58e+02]	5.78e+04	[3.12e+04, 9.23e+04]
Serial-fixed	DDPG	-6.48e+02	[-1.34e+03, -2.51e+02]	3.06e+04	[1.61e+04, 5.22e+04]
Serial-fixed	PPO	-6.46e+01	[-9.56e+01, -3.68e+01]	4.08e+03	[2.30e+03, 6.14e+03]
Serial-fixed	SAC	-1.51e+03	[-2.44e+03, -8.00e+02]	9.03e+04	[5.18e+04, 1.33e+05]
Serial-fixed	TD3	-1.53e+03	[-2.69e+03, -7.61e+02]	8.31e+04	[5.05e+04, 1.23e+05]
NVDEx	A2C	-5.60e+03	[-1.33e+04, -1.06e+03]	5.94e+02	[1.34e+02, 1.33e+03]
NVDEx	DDPG	-1.46e+04	[-3.13e+04, -3.37e+03]	3.22e+03	[4.76e+02, 7.39e+03]
NVDEx	PPO	-1.58e+01	[-3.07e+01, -4.57e+00]	2.97e+00	[7.01e-01, 5.82e+00]
NVDEx	SAC	-7.76e+03	[-1.56e+04, -1.15e+03]	1.56e+03	[1.13e+02, 3.13e+03]
NVDEx	TD3	-1.22e+04	[-2.24e+04, -4.20e+03]	2.78e+03	[6.96e+02, 5.67e+03]

normalized by the corresponding optimum. The resulting variance-ratio test on relative suboptimality is reproduced in Table 1.

This result clarifies why converse-generated continuous-time families are useful. The scientific question is not whether one optimizer attains a slightly higher final reward on one benchmark, but whether the learning pipeline remains *consistent* across nearby environments. Standard leaderboards usually hide this effect because they do not normalize by the task-specific optimum and do not treat environment variation itself as the object of study.

7.2 Native discrete-time fixtures: oracle-referenced comparison

The second use of the framework is oracle-referenced evaluation on native discrete-time fixtures. Here the key advantage is not just that the optimum is known, but that the benchmark is defined directly in discrete time, so the comparison is not confounded by an external integration step.

Table 2 reproduces the main overall-performance summary for three benchmark regimes: *serial-random*, *serial-fixed*, and *NVDEx*. The metrics are mean optimality gap and mean regret with paired-bootstrap 95% confidence intervals under CRN evaluation.

Several patterns emerge.

No universal winner. Different benchmark regimes reveal different failure modes. On serial-random configurations, where the initial-state schedule covers a broader region of state space,

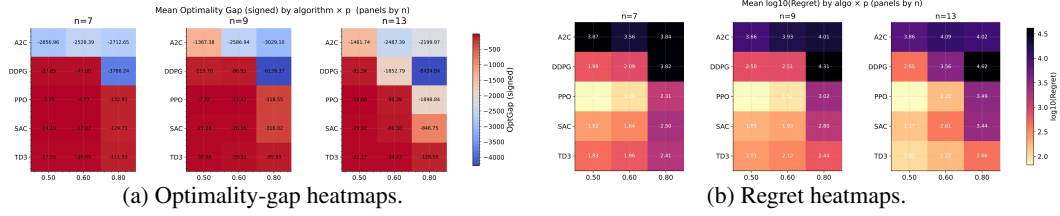


Figure 2: Oracle-referenced performance on converse-generated serial planar arm fixtures. Rows correspond to algorithms, columns to control-authority levels p , and panels to dimensionality. The point is not only which method ranks first, but how far each method remains from the certified optimum as difficulty parameters change.

240 TD3 and SAC benefit from replay-based coverage and attain the smallest regret. On serial-fixed
 241 configurations and on NVDEX, PPO is strongest, which is consistent with the fact that its constrained
 242 on-policy updates are comparatively stable in narrow occupancy distributions and in long-horizon
 243 stiff dynamics.

244 **Absolute metrics expose mechanism, not only rank.** Because V^* and π^* are known, the bench-
 245 mark distinguishes between being *better than another baseline* and being *actually close to optimal*.
 246 This matters because some methods can achieve reasonable reward yet still remain far from the oracle
 247 in gap or regret. In converse-optimal fixtures, such discrepancies are visible rather than hidden.

248 **Control authority is a meaningful difficulty knob.** As the continuation parameter p increases, the
 249 oracle policy becomes more effective at dissipating state cost, so the optimal value improves rapidly.
 250 Yet learned policies often fail to improve at the same rate. Consequently, regret can increase with p
 251 even when raw reward looks better. Stronger control makes the optimum easier to exploit but also
 252 amplifies policy imperfections, bootstrapping errors, and approximation mismatch.

253 **Why discrete time sharpens the interpretation.** For these results, the transition law used by the
 254 learner, the evaluator, and the oracle is identical. Observed differences are therefore attributable to the
 255 algorithm, the benchmark regime, and the controlled parameters $(n, p, \Sigma, \dots)$ —not to an additional
 256 choice of numerical solver step.

257 7.3 What the combined picture says

258 Taken together, the two kinds of results support a sharper view of benchmarking:

- 259 • Continuous-time converse generators are useful when the scientific question is about *sensitivity*—
 260 for example, whether changing an optimizer, architecture, or protocol alters the variance or mean
 261 of relative suboptimality across nearby plants.
- 262 • Native discrete-time converse fixtures are useful when the question is about *absolute policy*
 263 *quality*—for example, whether an RL algorithm is close to the known optimum, how regret scales
 264 with control authority, or which failures are due to approximation rather than exploration.

265 This is why keeping both continuous-time and discrete-time constructions is not redundant. They are
 266 not two versions of the same benchmark; they answer different empirical questions.

267 8 Discussion and Limitations

268 The central advantage of the framework is calibration. Once (V^*, π^*) is known, benchmark conclu-
 269 sions can be decomposed: a poor result may stem from critic error, policy error, insufficient coverage,
 270 sensitivity to environment shifts, or an unfavorable interaction between dynamics and optimizer. This
 271 is much more informative than a single return number.

272 At the same time, converse-optimal benchmarks are deliberately structured. The deterministic
 273 continuous-time construction assumes smoothness and control-affine dynamics. The discrete-time
 274 stochastic construction studied here uses control-affine systems, additive Gaussian noise, and a

275 Quadratic–Gaussian specialization to obtain explicit policies and values. These assumptions are
276 not meant to exhaust the space of RL environments; they are the price paid for exact optimality
277 certificates.

278 The scope is also intentionally limited. The benchmark families studied here are low-dimensional
279 state-space control problems, not pixel-based partially observable tasks. That restriction is deliberate:
280 the aim is calibration and diagnosis, not ecological breadth. Strong performance on these families
281 should therefore be interpreted as evidence about optimization, approximation, and benchmark
282 sensitivity under known optima—not as a claim of immediate superiority on all modern RL regimes.

283 A related limitation is interpretational. Continuous-time converse benchmarks are physically mean-
284 ingful, but empirical RL results on them depend on the chosen discretization and integration method.
285 We view this not as a defect, but as part of what these benchmarks reveal: they make it possible to
286 study how much of a conclusion is about the learning algorithm and how much is about the numerical
287 interface through which the algorithm experiences the plant. Native discrete-time fixtures, in turn,
288 solve that issue by construction, but they may abstract away some continuous-time physics. The two
289 views are therefore complementary.

290 Future work should extend the generator class in three directions: beyond Gaussian noise to heavy-
291 tailed or temporally correlated disturbances; beyond control-affine systems to more general action
292 couplings; and beyond quadratic value classes to richer certified optima. Another important direction
293 is partial observability, where converse-optimal benchmarking could help disentangle representation
294 error from control error.

295 9 Conclusion

296 We presented a ground-truth-first benchmark view of reinforcement learning based on converse
297 optimality. The framework supports two distinct but complementary benchmark modes. First,
298 converse-generated environment families enable statistically rigorous studies of sensitivity and
299 robustness. Second, certified fixtures with known (V^*, π^*) enable absolute algorithm comparison
300 by optimality gap, regret, Bellman residual, and policy error. The main conceptual point is that
301 continuous-time and discrete-time benchmarks should not be treated as interchangeable. Continuous-
302 time generators are especially useful for sensitivity studies, precisely because realized behavior
303 depends on discretization and numerical choices. Native discrete-time generators are especially
304 useful for oracle-referenced comparison, precisely because they remove that extra confound. By
305 combining both views, converse optimality provides a calibrated complement to score-driven RL
306 benchmarking.

307 References

- 308 [1] Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron Courville, and Marc G. Belle-
309 mare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in*
310 *Neural Information Processing Systems*, 2021.
- 311 [2] Marc G. Bellemare, Yavar Naddaf, Joel Veness, and Michael Bowling. The Arcade Learning
312 Environment: An evaluation platform for general agents. *Journal of Artificial Intelligence*
313 *Research*, 47:253–279, 2013.
- 314 [3] Richard Bellman. *Dynamic Programming*. Princeton University Press, 1957.
- 315 [4] Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang,
316 and Wojciech Zaremba. OpenAI Gym. arXiv:1606.01540, 2016.
- 317 [5] Maxime Chevalier-Boisvert, Bolun Dai, Mark Towers, Rodrigo Perez-Vicente, Lucas Willems,
318 Suman Pal, Pablo Samuel Castro, Jordan Terry, et al. Minigrid & Miniworld: Modular and
319 customizable reinforcement learning environments for goal-oriented tasks. In *Advances in*
320 *Neural Information Processing Systems*, 2023.
- 321 [6] Karl Cobbe, Christopher Hesse, Jacob Hilton, and John Schulman. Leveraging procedural
322 generation to benchmark reinforcement learning. In *Proceedings of the 37th International*
323 *Conference on Machine Learning*, 2020.

- 324 [7] Justin Fu, Aviral Kumar, Ofir Nachum, George Tucker, and Sergey Levine. D4RL: Datasets for
325 deep data-driven reinforcement learning. *arXiv:2004.07219*, 2020.
- 326 [8] Thomas Göhr, Pavel Osinenko, and Stefan Streif. Converse optimality for discrete-time systems.
327 *IEEE Transactions on Automatic Control*, 65(5):2257–2264, 2020.
- 328 [9] Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David
329 Meger. Deep reinforcement learning that matters. In *Proceedings of the AAAI Conference on*
330 *Artificial Intelligence*, 2018.
- 331 [10] Shengyi Huang, Quentin Gallouédec, Florian Felten, Antonin Raffin, Rousslan F. J. Dossa,
332 Yanxiao Zhao, Ryan Sullivan, Viktor Makovychuk, Denys Makoviichuk, Mohamad H. Danesh,
333 et al. Open RL Benchmark: Comprehensive tracked experiments for reinforcement learning.
334 *arXiv:2402.03046*, 2024.
- 335 [11] Sinan Ibrahim, Grégoire Ouerdane, Hadi Salloum, Henni Ouerdane, Stefan Streif, and Pavel
336 Osinenko. Benchmarking reinforcement learning via stochastic converse optimality: Generating
337 systems with known optimal policies. *arXiv:2603.17631*, 2026.
- 338 [12] Scott M. Jordan, Adam White, Bruno C. da Silva, Martha White, and Philip S. Thomas.
339 Position: Benchmarking is limited in reinforcement learning research. In *Proceedings of the*
340 *41st International Conference on Machine Learning*, 2024.
- 341 [13] Vesna Nevistic and James A. Primbs. Constrained nonlinear optimal control: A converse HJB
342 approach. Technical Report CIT-CDS 96-021, California Institute of Technology, 1996.
- 343 [14] Ian Osband, Yotam Doron, Matteo Hessel, John Aslanides, Eren Sezener, Andre Saraiva,
344 Katrina McKinney, Tor Lattimore, Csaba Szepesvári, Satinder Singh, Benjamin Van Roy,
345 Richard Sutton, David Silver, and Hado Van Hasselt. Behaviour Suite for reinforcement
346 learning. In *International Conference on Learning Representations*, 2020.
- 347 [15] Andrew Patterson, Samuel Neumann, Martha White, and Adam White. Empirical design in
348 reinforcement learning. *Journal of Machine Learning Research*, 25:1–63, 2024.
- 349 [16] James A. Primbs and Vesna Nevistic. Optimality of nonlinear design techniques: A converse
350 HJB approach. Technical Report CIT-CDS 96-022, California Institute of Technology, 1996.
- 351 [17] Martin L. Puterman. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*.
352 Wiley, 1994.
- 353 [18] Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based
354 control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2012.
- 355 [19] Grigory Yamerenko, Sinan Ibrahim, Francisco Moreno-Mora, Pavel Osinenko, and Stefan Streif.
356 Generating informative benchmarks for reinforcement learning. *IEEE Control Systems Letters*,
357 9:480–485, 2025.